Activity 5: Create and List CRM Contacts with Razor Pages

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO3 · K3/A3

Duration: 65 minutes

Runnable snapshot: src/LocalCRM

Goal

Implement page-focused create/list handlers through an injectable contact store.

What you will produce

input model, IContactStore boundary, create/list PageModels and successful PRG navigation

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Implement REQ-001 and REQ-002 with Razor Pages. Introduce IContactStore and an in-memory implementation registered as singleton for this temporary Activity only. Use a dedicated NewContact input model and Post/Redirect/Get. Limit changes to Models, Services, Pages/Contacts and Program.cs.

Step-by-step

Step 1 — Create the domain/input types

Use separate input and output shapes to prevent overposting.

Create or update LocalCRM/Models/CustomerContact.cs:

```
namespace LocalCRM.Models;
public sealed record CustomerContact(int Id, string Name, string Email, string Stage,
DateTimeOffset CreatedAt);
public sealed record NewContact(string Name, string Email, string Stage);
```

Step 2 — Declare the service boundary

Keep HTTP details out of the store contract.

Create or update LocalCRM/Services/IContactStore.cs:

```
using LocalCRM.Models;
namespace LocalCRM.Services;
public interface IContactStore
{
 Task<int> AddAsync(NewContact input, CancellationToken ct);
 Task<IReadOnlyList<CustomerContact>> ListAsync(CancellationToken ct);
}
```

Step 3 — Implement temporary memory storage

Use locking/concurrent collections or a reviewed implementation; make restart loss explicit.

Step 4 — Register the service

Add the service before builder.Build().

Create or update Program.cs registration:

```
builder.Services.AddSingleton<IContactStore, InMemoryContactStore>();
```

Step 5 — Build the Create PageModel

Bind a dedicated input model, guard ModelState, call the store and redirect to Index.

Step 6 — Build the form markup

Use asp-for labels, inputs and validation messages. Keep the Form Tag Helper enabled for antiforgery.

Step 7 — Load the list

Inject IContactStore into IndexModel and call ListAsync(HttpContext.RequestAborted).

Step 8 — Verify PRG

Submit a valid contact, observe 302 to /Contacts, then refresh.

```
curl -i -c cookies.txt -b cookies.txt http://localhost:5050/Contacts/Create
```

Expected evidence: Browser submission redirects; refresh does not create a duplicate.

Step 9 — Verify restart limitation

Restart the app and confirm the temporary list is empty; record why Activity 8 replaces it.

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5105
```

Open http://localhost:5105. For Activities 7–10, also probe http://localhost:5105/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.
- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.

- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO3 and K3/A3.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	